

MATH4996 Option Pricing and Early Exercise Boundary of American Options Using Trinomial Tree Model

Wong King Long, Ken

9th October 2025

1 Introduction to American Options

American options differ from European options by allowing early exercise before expiration. This feature adds an early exercise premium to their value compared to European counterparts. This report discusses pricing properties, optimal exercise policies, and formulations for American calls and puts on dividend-paying assets.

Key points:

- Early exercise of an American call on a non-dividend-paying asset is never optimal, equating its value to the European call.
- For dividend-paying assets, early exercise may be optimal for calls when the asset price is sufficiently high, and always for puts when the interest rate is positive.
- The optimal exercise boundary separates the continuation region (hold the option) from the stopping region (exercise).

2 Optimal Exercise Boundaries

The optimal exercise boundary $S^*(\tau)$ where τ is time to maturity, is determined as part of the pricing problem.

2.1 American Calls with Continuous Dividend Yield

For an American call with dividend yield $q > 0$:

- Early exercise is optimal when $S \geq S^*(\tau)$.
- $S^*(\tau)$ is continuous and increasing in τ .
- Near expiration ($\tau \rightarrow 0^+$): $S^*(0^+) = K \max(1, r/q)$, where K is the strike price.
- At infinity ($\tau \rightarrow \infty$): $S_\infty^* = \mu_+ / (\mu_+ - 1)K$, where μ_+ is the positive root of the auxiliary equation.
- Higher volatility σ typically lowers $S^*(\tau)$, making early exercise less likely as the option's time value increases.

2.2 American Puts with Continuous Dividend Yield

For an American put:

- Early exercise is optimal when $S \leq S^*(\tau)$.
- $S^*(\tau)$ is continuous and decreasing in τ .
- Near expiration: $S^*(0^+) = X \min(1, r/q)$.
- At infinity: $S_\infty^* = \mu_- / (\mu_- - 1)X$, where μ_- is the negative root.
- Higher σ raises $S^*(\tau)$, expanding the exercise region.

Additional information: Put-call symmetry relates call and put boundaries: $S_C^*(\tau; r, q) = K^2 / S_P^*(\tau; q, r)$.

3 Relation to Python Script

The provided Python code implements a trinomial tree model to price American options and compute exercise boundaries:

- The *TrinomialProbs* function calculates probabilities matching moments for the log-asset price process.
- *american_call_boundary* and *american_put_boundary* build the tree backward from maturity, computing option values as the maximum of intrinsic value and continuation value.
- The boundary is extracted by finding, at each time step, the minimum (for calls) or maximum (for puts) stock price where the option value equals the intrinsic value.
- This numerical method approximates the free boundary problem described in the chapter, where the boundary is not known a priori but emerges from the max condition.

The code assumes continuous dividend yield, aligning with the chapter's Section 5.1.1.

4 Theoretical Meaning of the Graphs

The graphs plot exercise boundaries against time, with parameters $S_0 = 60$, $r = 0.03$, $q = 0.05$, varying σ , and strikes $K = 55$ (call), $K = 65$ (put).

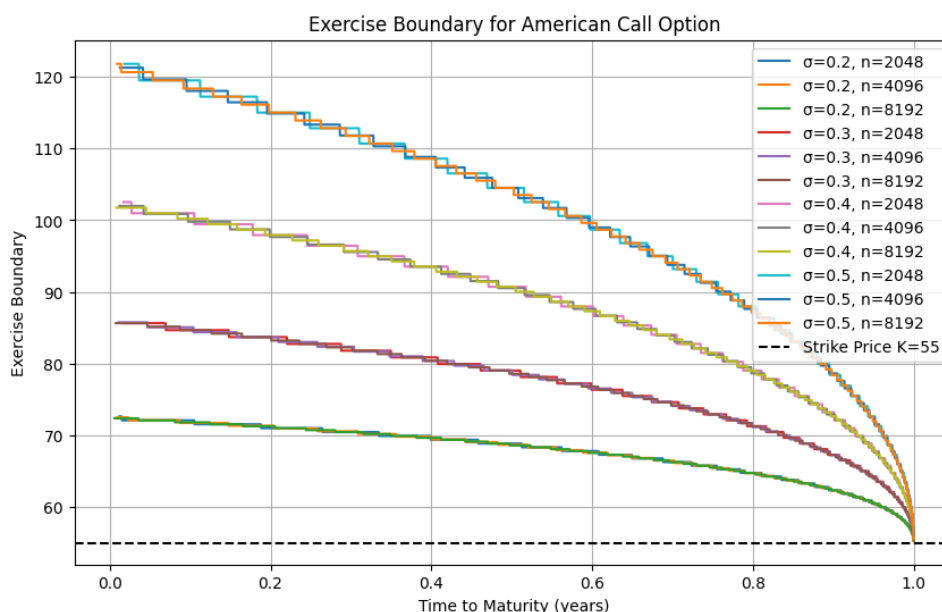


Figure 1: Exercise Boundary for American Call Option

Call Graph: As shown in Figure 1, boundaries decrease over the x-axis. If labeled "Time to Maturity," this would contradict theory (should increase with τ); likely, it represents time from now (current to maturity), so decreasing as maturity approaches, consistent with $S^*(\tau)$ increasing in τ . Higher σ lowers the boundary, reflecting increased option value delaying exercise.

Put Graph: As shown in Figure 2, boundaries increase over the x-axis, consistent with $S^*(\tau)$ decreasing in τ if x is time from now. Higher σ raises the boundary, expanding the exercise region. These behaviors stem from trade-offs: for calls, dividends vs. time/insurance value loss; for puts, interest gain vs. insurance loss. Higher volatility enhances holding value, altering boundaries as shown.

References

- [1] Kwok, Y.K. *Mathematical Models of Financial Derivatives*. Springer, 2007. https://www.math.hkust.edu.hk/~maykwok/courses/ma571/06_07/Kwok_Chap_5.pdf

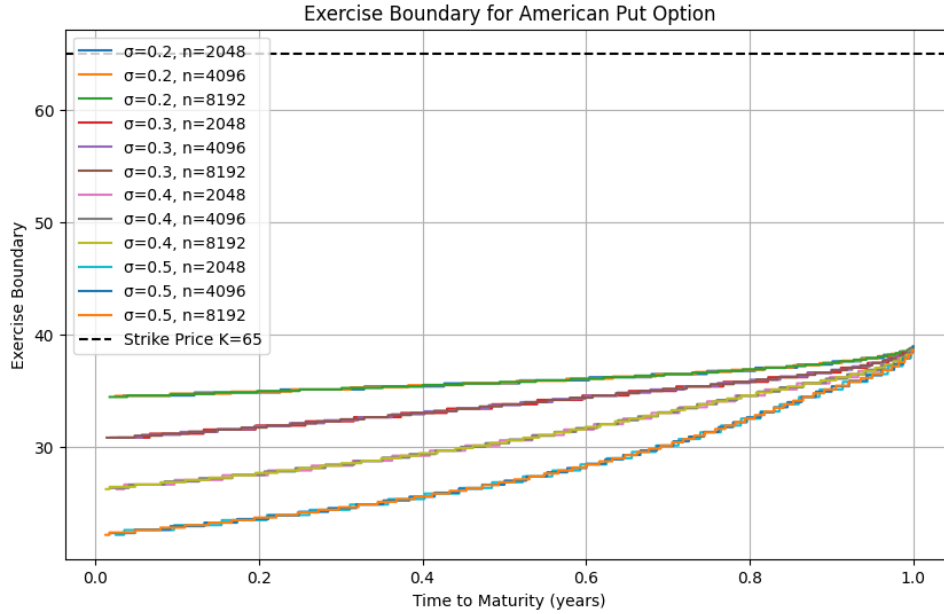


Figure 2: Exercise Boundary for American Put Option

Appendix: Code Implementation

```
# -*- coding: utf-8 -*-
"""MATH4999_TrinomialTreeAmOptions_2nd.ipynb

Automatically generated by Colab.

Original file is located at
https://colab.research.google.com/drive/1AcAJc56ssrg0EzVCG1BavUExHuIDU6bc
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import time

def TrinomialProbs(r, q, sigma, deltaT, lambda_val=np.sqrt(3)):
    """
    Calculate trinomial tree probabilities using optimal lambda for kurtosis matching
    Solves: 1) Mean matching, 2) Variance matching, 3) Probabilities sum to 1
    """
    nu = r - q - 0.5 * sigma**2
    deltaX = lambda_val * sigma * np.sqrt(deltaT)

    # System of equations solution
    V = sigma**2 * deltaT + nu**2 * deltaT**2

    p_u = 0.5 * (V / deltaX**2 + nu * deltaT / deltaX)
    p_d = 0.5 * (V / deltaX**2 - nu * deltaT / deltaX)
    p_m = 1 - p_u - p_d

    return p_u, p_m, p_d, deltaX, lambda_val

S0 = 60
K_call = 55
K_put = 65
r = 0.03
q = 0.05
T = 1.0
sigma_values = [0.2, 0.3, 0.4, 0.5]
n_values = [2**11, 2**12, 2**13]

def american_call_boundary(S0, K, r, q, sigma, T, N):
    deltaT = T / N
    p_u, p_m, p_d, deltaX, lam = TrinomialProbs(r, q, sigma, deltaT)
    discount = np.exp(-r * deltaT)

    # Set up S values (at maturity)
    Svals = S0 * np.exp(np.arange(N, -N-1, -1) * deltaX)

    # Set up lattice and terminal values
```

```

Cvals = np.zeros((2 * N + 1, N + 1))
Cvals[:, N] = np.maximum(Svals - K, 0)

# Backward induction vectorized
for j in range(N - 1, -1, -1):
    i_vals = np.arange(N - j, N + j + 1)
    hold = discount * (p_u * Cvals[i_vals - 1, j + 1] +
                       p_m * Cvals[i_vals, j + 1] +
                       p_d * Cvals[i_vals + 1, j + 1])
    Cvals[i_vals, j] = np.maximum(Svals[i_vals] - K, hold)

# Compute exercise boundary vectorized
boundary = np.full(N, np.nan)
for j in range(N - 1, -1, -1):
    i_vals = np.arange(N - j, N + j + 1)
    mask = (Cvals[i_vals, j] == Svals[i_vals] - K)
    if np.any(mask):
        boundary[j] = np.min(Svals[i_vals][mask]) # min for call

return (T, boundary)

# Generate exercise boundary for different values of sigma and n
exercise_boundary_call = {}
for sigma in sigma_values:
    for n in n_values:
        exercise_boundary_call[(sigma, n)] = american_call_boundary(S0, K_call, r, q, sigma, T, n)

# Plotting the exercise boundary
plt.figure(figsize=(10, 6))
for sigma in sigma_values:
    for n in n_values:
        n1, prices = exercise_boundary_call[(sigma, n)]
        deltaT = T / n
        times_from_now = np.arange(0, n) * deltaT
        plt.plot(times_from_now, prices, label=f"={sigma}, n={n}")
plt.axhline(y=K_call, linestyle='--', color='k', label='Strike Price-K=55')
plt.xlabel('Time-to-Maturity-(years)')
plt.ylabel('Exercise-Boundary')
plt.title('Exercise-Boundary-for-American-Call-Option')
plt.legend()
plt.grid(True)
plt.show()

def american_put_boundary(S0, K, r, q, sigma, T, N):
    deltaT = T / N
    p_u, p_m, p_d, deltaX, lam = TrinomialProbs(r, q, sigma, deltaT)
    discount = np.exp(-r * deltaT)

    # Set up S values (at maturity)
    Svals = S0 * np.exp(np.arange(N, -N-1, -1) * deltaX)

    # Set up lattice and terminal values
    Pvals = np.zeros((2 * N + 1, N + 1))
    Pvals[:, N] = np.maximum(K - Svals, 0)

    # Backward induction vectorized
    for j in range(N - 1, -1, -1):
        i_vals = np.arange(N - j, N + j + 1)
        hold = discount * (p_u * Pvals[i_vals - 1, j + 1] +
                           p_m * Pvals[i_vals, j + 1] +
                           p_d * Pvals[i_vals + 1, j + 1])
        Pvals[i_vals, j] = np.maximum(K - Svals[i_vals], hold)

    # Compute exercise boundary vectorized
    boundary = np.full(N, np.nan)
    for j in range(N - 1, -1, -1):
        i_vals = np.arange(N - j, N + j + 1)
        mask = (Pvals[i_vals, j] == K - Svals[i_vals])
        if np.any(mask):
            boundary[j] = np.max(Svals[i_vals][mask]) # max for put

    return (T, boundary)

# Generate exercise boundary for different values of sigma and n
exercise_boundary_put = {}
for sigma in sigma_values:
    for n in n_values:
        exercise_boundary_put[(sigma, n)] = american_put_boundary(S0, K_put, r, q, sigma, T, n)

# Plotting the exercise boundary
plt.figure(figsize=(10, 6))
for sigma in sigma_values:
    for n in n_values:
        n1, prices = exercise_boundary_put[(sigma, n)]
        deltaT = T / n
        times_from_now = np.arange(0, n) * deltaT
        plt.plot(times_from_now, prices, label=f"={sigma}, n={n}")

```

```
plt.axhline(y=K_put, linestyle='—', color='k', label='Strike Price K=65')
plt.xlabel('Time to Maturity (years)')
plt.ylabel('Exercise Boundary')
plt.title('Exercise Boundary for American Put Option')
plt.legend()
plt.grid(True)
plt.show()
```